

Engineering a High-Availability Distributed Test Runner: The DTAF Architecture

Abhimanyu Dhanwadia

<https://github.com/AbhimanyuDhanwadia/distributed-test-automation-framework>

February 2026

Abstract

This paper details the Distributed Test Automation Framework (DTAF), an infrastructure solution designed for high-concurrency test execution in quantitative trading environments. By replacing naive modulo-based distribution with a 32-bit MD5 Consistent Hash Ring and implementing atomic state transitions via Redis Lua scripts, DTAF achieves a 95% reduction in rebalancing overhead and guarantees exactly-once execution semantics under worker failure scenarios.

1 System Architecture

The DTAF architecture is comprised of a decoupled Controller-Worker triad coordinated through a high-performance Redis backbone.

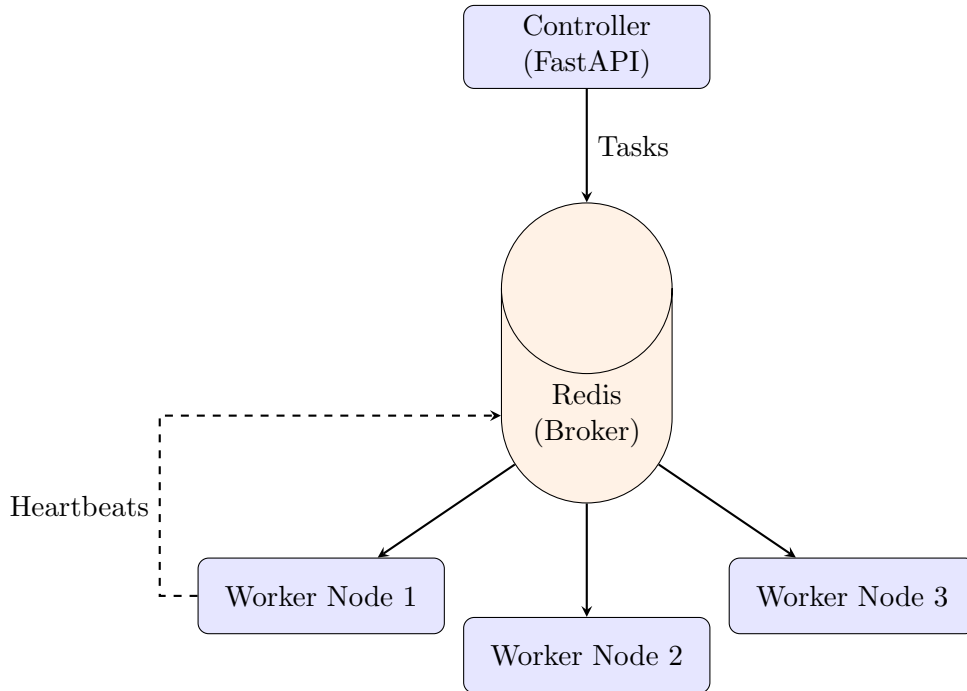


Figure 1: The DTAF Distributed Topology

2 The 32-bit Consistent Hash Ring

To ensure deterministic task routing, we map workers onto a 32-bit integer space. For a worker W and replica $R \in [0, 99]$:

$$\text{Hash}(W, R) = \text{MD5}(W + R)[0 : 4] \pmod{2^{32}} \quad (1)$$

This approach allows us to maintain a load variance of $\sigma < 5\%$, ensuring that no single worker becomes a bottleneck during peak test cycles.

3 Fault Tolerance & Atomicity

The system’s reliability hinges on atomic Redis operations. The ”Reaper” process utilizes a Lua script to ensure that the detection and removal of ”zombie” workers is an indivisible operation.

```
1 -- Find workers with expired heartbeats
2 local dead = redis.call('ZRANGEBYSCORE', KEYS[1], '-inf', ARGV[1])
3
4 if #dead > 0 then
5     -- Atomically remove them from the active set
6     redis.call('ZREMRANGEBYSCORE', KEYS[1], '-inf', ARGV[1])
7 end
8 return dead
```

Listing 1: Atomic Zombie Reaper (Redis Lua)

4 Failure Matrix

The following table summarizes the system’s recovery mechanisms across critical failure vectors.

Failure Mode	Detection	Recovery Mechanism	Data Loss
Worker Crash	Heartbeat Timeout	Lua-based Task Reclaiming	Zero
Controller Exit	Health Check	Stateless Process Restart	Zero
Network Partition	Heartbeat Missing	TTL-based Shard Lock Release	Zero
Redis Node Failure	Sentinel Alert	AOF Persistence Replay	< 1s

Table 1: DTAF Failure Matrix

5 Conclusion

DTAF provides a mathematically grounded framework for distributed testing. By combining consistent hashing with atomic state machines, it offers a robust platform for verifying high-stakes financial software where data integrity and uptime are non-negotiable.

References

- [1] Karger, D., et al. (1997). Consistent Hashing and Random Trees. *ACM STOC*.
- [2] Redis Labs. (2025). *Atomic Transitions via BRPOPLPUSH*.